	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	1289 - floatToHalfKernel	(884736, 1, 1)x(256, 1, 1)	10.78 ms	20,704,778	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.92 Ghz	[35348] MPI.exe	

Summary | Details | Source | Context | Comments | Raw | Session

Compare | Tools | View | Export

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	61.96	Duration [ms]	10.78
Memory Throughput [%]	49.62	Elapsed Cycles [cycle]	20,704,778
L1/TEX Cache Throughput [%]	22.67	SM Active Cycles [cycle]	20,702,478.62
L2 Cache Throughput [%]	20.08	SM Frequency [Ghz]	1.92
DRAM Throughput [%]	49.62	DRAM Frequency [Ghz]	7.99

High Compute Throughput Compute is more heavily utilized than Memory: Look at the Compute Workload Analysis section to see what the compute pipelines are spending their time doing. Also, consider whether any computation is redundant and could be reduced or moved to look-up tables.

Key Performance Indicators

Roofline Analysis The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved 0% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the Profiling Guide for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [us]	4	# Pass Groups	3
Maximum Buffer Size [Mbyte]	11.86	-	-

Compute Workload Analysis

Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	1.67	SM Busy [%]	61.96
Executed Ipc Active [inst/cycle]	1.67	Issue Slots Busy [%]	41.66
Issued Ipc Active [inst/cycle]	1.67		

High Utilization ALU is the highest-utilized pipeline (62.0%) based on elapsed cycles in the workload, taking into account the rates of its different instructions. It executes integer and logic operations. The pipeline is well-utilized, but might become a bottleneck if more work is added. Based on the number of executed instructions, the highest utilized pipeline (62.0%) is ALU. It executes integer and logic operations. Comparing the two, the overall pipeline utilization appears to be caused by frequent, low-latency instructions. See the Profiling Guide or hover over the pipeline name to understand the workloads handled by each pipeline. The Instruction Statistics section shows the mix of executed instructions for this workload. Check the Warp State Statistics section for which reasons cause warps to stall.

Key Performance Indicators

Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	126.92	Mem Busy [%]	12.52
L1/TEX Hit Rate [%]	62.32	Max Bandwidth [%]	49.62
L2 Hit Rate [%]	59.56	Mem Pipes Busy [%]	39.88
L2 Compression Input Sectors [sector]	0	Local Memory Spilling Requests	0
L2 Compression Ratio	0	Shared Memory Spilling Requests	0
L2 Compression Success Rate [%]	0	Local Memory Spilling Request Overhead [%]	0
L2 Persisting Size [Mbyte]	6.29	Shared Memory Spilling Request Overhead [%]	0
L2 Sector Promotion Misses [%]	0	-	-

Local Memory Usage
Est. Speedup: 20.03%

This workload accesses local memory, accounting for 57.14% of all sectors requested in L1TEX. Local memory refers to a memory space that is private to each thread. Although it is logically distinct, local memory resides in global memory. This can lead to higher memory latency compared to the usage of registers or shared memory, a decrease in available bandwidth to global memory, and an increase in executed instructions.
The local memory accesses of this workload are largely cached in L1TEX (99.79% of loads, and 99.79% of stores). However, even spilling to L1TEX might lead to high performance penalties compared with staying in registers.
One common reason for the use of local memory is arrays being too large to fit into registers or not being indexed using compile time constants (see the "Local Memory" section of the CUDA Programming Guide. For more advice on how to reduce the usage of local memory refer to the documentation.

Key Performance Indicators

L1TEX Local Store Access Pattern
Est. Speedup: 10.98%

The memory access pattern for local stores to L1TEX might not be optimal. On average, only 16.5 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced local stores.

Key Performance Indicators

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	10.91	No Eligible [%]	58.35
Eligible Warps Per Scheduler [warp]	0.51	One or More Eligible [%]	41.65
Issued Warp Per Scheduler	0.42		

Issue Slot Utilization
Est. Local Speedup: 38.04%

Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 2.4 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this workload allocates an average of 10.91 active warps per scheduler, but only an average of 0.51 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the Warp State Statistics and Source Counters sections can help, too.

Key Performance Indicators

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	26.20	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	26.20	Avg. Not Predicated Off Threads Per Warp	31.73

Wait Stalls
Est. Speedup: 38.04%

On average, each warp of this workload spends 13.7 cycles being stalled waiting on a fixed latency execution dependency. Typically, this stall reason should be very low and only shows up as a top contributor in already highly optimized kernels. Try to hide the corresponding instruction latencies by increasing the number of active warps, restructuring the code or unrolling loops. Furthermore, consider switching to lower-latency instructions, e.g. by making use of fast math compiler options. This stall type represents about 52.4% of the total average of 26.2 cycles between issuing two instructions.

Key Performance Indicators

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Profiling Guide provides more details on each stall reason.

Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	828,112,896	Avg. Executed Instructions Per Scheduler [inst]	8,626,176
Issued Instructions [inst]	828,115,200	Avg. Issued Instructions Per Scheduler [inst]	8,626,200
Local Memory Spilling Requests [inst]	0	Shared Memory Spilling Requests [inst]	0

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	884,736	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	24	Static Shared Memory Per Block [byte/block]	0
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	226,492,416	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	6,144	Shared Memory Configuration Size [Kbyte]	16.38
Uses Green Context	0	Stack Size	1,024
# SMs [SM]	24	# TPCs	12
Enabled TPC IDs	all	-	-

Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	10
Theoretical Active Warps per SM [warp]	48	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	91.56	Block Limit Warps [block]	6
Achieved Active Warps Per SM [warp]	43.95	Block Limit SM [block]	24

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	20,702,478.62	Average L1 Active Cycles [cycle]	20,702,478.62
Average L2 Active Cycles [cycle]	18,921,065.06	Average SMSP Active Cycles [cycle]	20,711,166.97
Average DRAM Active Cycles [cycle]	42,770,456	Total SM Elapsed Cycles [cycle]	496,914,392
Total L1 Elapsed Cycles [cycle]	496,914,392	Total L2 Elapsed Cycles [cycle]	315,746,880
Total SMSP Elapsed Cycles [cycle]	1,987,657,568	Total DRAM Elapsed Cycles [cycle]	344,777,728

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	56,623,104	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.07	Avg. Divergent Branches [branches]	0

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.